

Priced Plausible Fiction

A Two-Sided Market of Programmatic Intentions on the
Cost-Accounted Rho Calculus

F1R3FLY.io

in dialogue with D. Spivak (Topos Institute)

June 30, 2026 — Draft for discussion

Abstract

Plausible Fiction (PF) makes a marketplace out of typed holes: a participant publishes a runnable program fragment with holes that other participants fill, and a hole-free (*grounded*) fragment is dispatched back to its contributors for execution. The original design leaves the broker a type-filtered *forwarder* and explicitly defers *valuation* to a later layer. Independently, the cost-accounted rho calculus realises the Peyton Jones–Eber–Seward financial combinators as an *executable settlement language*, with located authority over funds, an acceptance gate that proves a deployment funded before any step fires, and atomicity by construction. This note fuses the two. We promote each agent hole from a guarded receiver to an *order book*: a *bid* is a holey PF carried with a reservation price and a located, signature-drawable purse; an *ask* is a candidate filler carried with a reservation price — and, because a filler carries its own holes, a supply PF is simultaneously *one ask plus a bid per residual hole*, so bid and ask alternate down the demand tree. The broker becomes a *matching engine* that clears asks against bids: the type guard decides *admissibility*, price decides the *clear*, and the cost calculus’s acceptance gate makes each clear atomic and each grounded tree fully funded before it runs. The valuation of a priced PF is the Peyton Jones–Eber–Seward denotation read *over* the conditional demand tree: parallel frontiers sum and fund as an atomic join, branches the holder controls take a conservative bound with refund, scaling multiplies by an observable, *give* swaps bid/ask polarity, and run-time outcomes stage as options. Price lives in a preference quantale kept strictly apart from phlogiston, with a named seam connecting the two. We give the object-language and bridge deltas, the extended coordination protocol with escrow, a revised lifecycle, a requirements catalogue, a worked example (Alice’s venue, priced and cleared), and an honest accounting of what is established versus conjectural — in particular that the clearing *rule* is policy, not a theorem of the calculus, and that pricing a nature-resolved branch wants a probability layer.

Contents

1	Purpose and the unification	2
2	Conceptual model: holes as markets	3
2.1	Bid and ask at a hole	3
2.2	The matching engine clears; the type guard only admits	4
2.3	Netting is clearing	4
3	Architecture	4
4	Object language: pricing PFLam	4

5	Valuation: the PJES denotation over the demand tree	5
5.1	The price quantale	5
5.2	Valuation is structural recursion over the CDT	6
6	Two graded dimensions, kept apart	6
7	Bridge: two new folds	7
8	Coordination: the matching engine	7
8.1	Publish and arm: a market per frontier hole	8
8.2	Offer: a supply PF is one ask plus a bid per residual hole	8
8.3	Match and clear: atomic, type-guarded, refunding	8
8.4	Grounding, running, settlement	9
8.5	Templates carry price priors; caveats are price discovery	10
9	Lifecycle	10
10	Requirements catalogue	11
11	Worked examples: branching, then competition	12
12	Established vs. conjectural; open problems	12
A	A Worked Example with Competition	13
A.1	What the Alice example did not show	14
A.2	Setup: one type, two buyers, three fillers	14
A.2.1	The demanded type	14
A.2.2	Two competing holey contexts (the buyers)	14
A.2.3	Three competing fillers (the asks)	15
A.3	The feasible set: admit, then price	15
A.4	The books as data	15
A.5	Clearing: the rule decides who is served	16
A.6	The concurrent trace (greedy lowest-ask)	17
A.7	Settlement	18
A.8	What this example establishes	18
	Glossary	19

1 Purpose and the unification

This note specifies how to combine two existing designs into one venue. The first is *Plausible Fiction* (PF), a marketplace in which a participant publishes a PFLam term — a dependently typed λ -term with *typed holes* — and other participants supply PFs that fill those holes; a hole-free term is *grounded* and may be *run*, which dispatches each leaf obligation back to whichever participant supplied it. In that design the broker is a *forwarder* with an optional type filter, and value is explicitly deferred: “valuation is the next layer, and it does not live in the type . . . the same quantale-grading shape as phlogiston cost, valued in preference rather than resource.”

The second is the realisation of the Peyton Jones–Eber–Seward (PJES) financial combinators in the *cost-accounted rho calculus*: a settlement language whose multi-step operations are atomic by construction, whose authority over funds is an object capability carried by an unforgeable name, and whose deployments are admitted only after a linear resource proof certifies $\Sigma \geq \Delta$ (supply at least demand) for every signature, before any step fires.

The fusion is one idea applied uniformly:

Every agent hole is a two-sided market. The PF type guard decides which fillers are admissible; a reservation price and a located purse decide the clear; the cost calculus's acceptance gate makes each clear atomic and each grounded tree provably funded before it runs. The valuation of a priced PF is its PJES denotation, computed over its conditional demand tree.

Three commitments fix the design and are honoured throughout.

Polarity.

The publisher of a holey PF is the *buyer* (posts a *bid*); the supplier of a filler is the *seller* (posts an *ask*). Because a filler is itself holey, the supplier is simultaneously a buyer for its residual holes: *a supply PF is one ask plus a bid per residual hole*. Bid and ask therefore alternate down the demand tree, and the recursion that drove PF's marketplace (filling a hole can introduce new holes) is exactly the recursion that threads prices.

Price is not phlogiston.

"PF + cost" means PF carried with a *price/value* commitment, valued in a preference quantale $\mathcal{Q}$, kept strictly apart from the phlogiston (gas) that runs settlement. The cost paper warns that phlogiston counts funded rendezvous, not the magnitude of value moved; we keep the two graded dimensions separate and gate against their product. We name a seam (§6) at which the two may later be connected, but never conflate them.

Search stays in the market.

Checking $q \models T$ is decidable; *finding* a q is higher-order search and undecidable. As in PF, we do not run a solver: the type guard is only the acceptance test, and the order book — now a clearing engine rather than a forwarder — is the distributed, economically incentivised search procedure.

What is and is not re-specified. We take PFLam, its conditional demand tree (CDT), the cost-accounted rho calculus, the located token stack, and the acceptance gate as *given*. We add: a price annotation to the object language (§4); two bridge folds for valuation and clearing (§7); the matching engine, escrow, and settlement of the coordination layer (§8); the lifecycle, requirements, and worked example (§§9–11); and an honest open-problems section (§12).

2 Conceptual model: holes as markets

2.1 Bid and ask at a hole

In PF a hole's channel $@[u, \text{"demand"}, pfId, h]$ carries a single persistent guarded receiver: any candidate q with $q \models T$ is plugged. We replace that receiver with an *order book* holding two kinds of order.

Definition 1 (Bid). A *bid* at hole h is a tuple

$$\text{bid}(h) = (\Gamma_h \vdash T, \pi_b \in \mathcal{Q}, L_b),$$

where T is the (telescope-closed) hole interface, π_b is a *reservation price* (the buyer's maximum willingness to pay, valued in the price quantale $\mathcal{Q}$), and L_b is a *located purse* — a stack held on an unforgeable channel in the sense of the cost calculus — committing value up to π_b and the phlogiston needed to settle. The bid is the buyer's demand-side commitment: it says which fillers are admissible (T) and what it will pay for one (π_b , escrowed in L_b).

Definition 2 (Ask). An *ask* at hole h is a tuple

$$\text{ask}(h) = (q, \pi_a \in \mathcal{Q}, \text{addr}_a) \quad \text{with} \quad q \models T,$$

where q is an offered filler that type-checks against the hole interface, π_a is the seller’s reservation price (its minimum acceptable receipt), and addr_a is the ledger address at which payment lands. Since q may itself contain agent holes, posting this ask *also* posts a bid for each hole in $\text{holes}(q)$ ’s now-frontier: the seller is a buyer for what its own offer still needs.

The duality is exact and recursive. “PF + a cost” (Def. 1) is the buyer’s object; “a PF whose holes are priced” (Def. 2, unfolded one level) is the seller’s object — it offers q at π_a and prices, by attached bids, the holes q leaves open. A clear at h thus turns the seller’s residual now-frontier into the buyer’s next frontier, splicing two order books together exactly as plugging splices two CDTs in PF.

2.2 The matching engine clears; the type guard only admits

This is the load-bearing separation. Following PF, $q \models T$ is decidable on the normalising fragment and merely *admits* q into the feasible set. *Among admissible asks*, the engine clears by price: an ask is *feasible* for a bid when $q \models T$ and $\pi_a \leq \pi_b$ (the seller will accept no more than the buyer will pay), and among feasible asks the engine selects one by a clearing rule and settles at a clearing price $\pi_\star$ with $\pi_a \leq \pi_\star \leq \pi_b$. The clearing rule is a *policy parameter* (lowest ask; uniform-price double auction; the Dutch descent the cost paper already implements), not forced by the calculus (§12). What the calculus *does* force is that the clear is atomic: the value transfer, the plug, the provenance record, and the refund of $\pi_b - \pi_\star$ all happen under one acceptance gate, all-or-nothing.

2.3 Netting is clearing

The fusion immediately recovers a fact the cost paper proves by hand. `give` reverses the parties of a contract; in market terms it swaps bid and ask polarity, so the buyer of `give c` is the seller of c . A bilateral swap — a future `and(scale(spot, one), give(one F))` — is therefore a *matched pair* in which each party simultaneously bids (for the leg it receives) and asks (for the leg it gives). Crossing the two bids into a single net transfer is exactly the cost paper’s *netting*: the bilateral swap collapses to one atomic, single-token settlement in the data-dependent direction. *Netting is the market clearing the swap at its net*. What the cost paper installs as a hand-written optimisation, the priced market performs as its ordinary clearing operation.

3 Architecture

We keep PF’s three strata and extend two of them; the new *market* stratum sits where the forwarder broker used to be (Figure 1).

4 Object language: pricing PFLam

PFLam is unchanged as a calculus of meaning; we add only the means to *carry* a price and to read PJES structure off a term. A price is a value in the preference quantale $\mathcal{Q}$ (§5); the object language treats it opaquely (an element of a `Price` sort) and the bridge interprets it.

```
language! { // delta over PFLam (sec. 4 of the PF note); only additions shown
  types {
    Price      // an element of the preference quantale Q (opaque to PFLam)
    Obsv       // a time-varying observable: a persistent channel returning Q
  },
```

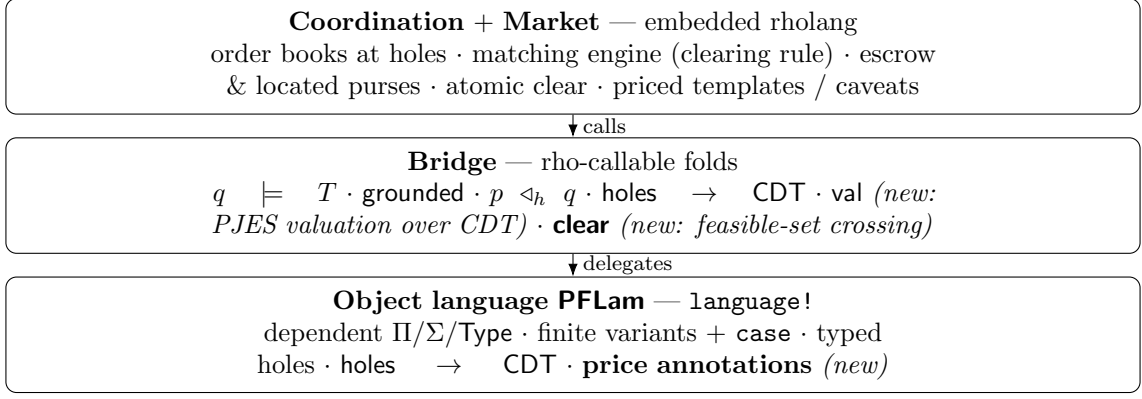


Figure 1: The strata. The market stratum replaces PF’s forwarder broker; two new bridge folds (val, clear) and a price annotation on the object language are the only additions to the lower layers. Object-language terms still ride on channels as payload.

```

terms {
  // A reservation price annotation on a hole: "this hole is offered/sought at p".
  // Pure: it rides with the hole; the bridge's val/clear interpret it.
  Priced . h:HoleId, ty:Term, p:Price |- "?" h ":" ty "@" p : Term;

  // Scale a (sub-)demand by an observable, exactly PJES scale(o,c).
  Scale . o:Obsv, c:Term |- "scale" "(" o "," c ")" : Term;

  // Reverse polarity: the PJES give. Swaps which side funds the leg, hence
  // swaps bid<->ask when the bridge prices it (sec. 2.3, 5.2).
  Give . c:Term |- "give" "(" c ")" : Term;

  // val(p): the PJES denotation of p as a demand price over its CDT (sec. 5).
  Val . p:Term |- "val" "(" p ")" : Term ![{
    crate::pflam::value_over_cdt(&p) // -> Term::Price(_)
  }] fold;
},
equations {
  // give is an involution; scaling by the constant-1 observable is identity.
  GiveInv . |- (give (give c)) = c ;
  Scale1 . |- (scale one_obs c) = c ;
},
}

```

Two points. First, Priced attaches a reservation price to a hole without changing its *type*: admissibility ($q \models T$) is untouched, so all of PF’s decidability and termination results carry over verbatim. Second, val is a pure fold: pricing is a total, deterministic computation over the term’s demand structure, performed by the bridge, never a search.

5 Valuation: the PJES denotation over the demand tree

5.1 The price quantale

Prices live in a *preference quantale* $\mathcal{Q} = (Q, \leq, \otimes, e)$: a commutative ordered monoid with $\otimes$ for *combining* concurrent demands (it will be addition of prices in the canonical instance) and $\leq$ for *ranking* (“cheaper”, or more generally “preferred”). This is the object PF §10 names for its valuation layer, $\text{val} : \text{Runs}(\text{CDT}) \rightarrow Q$, and it is the same grading shape as phlogiston cost — valued in preference, not resource. We instantiate Q as $(\mathbb{Q}_{\geq 0} \cup \{\infty\}, \leq, +, 0)$ for monetary clearing, but keep the algebra abstract so that “earlier is better” or “more turnout is better” preferences slot in unchanged.

5.2 Valuation is structural recursion over the CDT

The central identification: *pricing a PF is the PJES compositional valuation, with the combinator tree replaced by the conditional demand tree*. PJES value a contract by structural recursion against a model of observables; here the same recursion runs over the CDT, and the “value” it produces is the buyer’s required commitment — the price the bid must escrow. Write $\text{price}(\cdot) \in \mathcal{Q}$ for this functional. The clauses mirror PJES one-for-one (Table 1):

CDT shape	/ PFLam	PJES reading	Priced-market treatment
parallel $\{h_i : T_i\}$	now-frontier	$\text{and}(c_1, \dots, c_n)$	$\text{price} = \bigotimes_i \text{price}(h_i)$; funded as one <i>atomic join</i> (Pattern A): the bid escrows the sum, the gate reserves it all-or-nothing
wait-edge, chooses	holder	$\text{or}(c_1, \dots, c_n)$	conservative bound $\text{price} = \bigvee_i \text{price}(\text{CDT}_i)$ (max over branches); realized branch fixes spend, unforced side refunded (Pattern C)
wait-edge, solves (Obs)	nature re-	expectation $\mathbb{E}_\mu[\cdot]$	staged as an <i>option</i> : priced $\mathbb{E}_\mu[\text{price}(\text{CDT}_i)]$ under a model μ ; the distribution-free upper bound is the same max as above (§12)
scale(o, c)		scale(o, c)	$\text{price} = o \cdot \text{price}(c)$ (observable read at valuation / run time)
give(c)		reverse parties	swap bid/ask polarity: the buyer of give c is the seller of c
truncate/get/anytime		temporal combinators	the live branch at the realized time; anytime is an American-style option

Table 1: The valuation dictionary. Reading the PJES denotation over the CDT makes a priced PF *be* a combinator contract, and its valuation *be* its market price. The right column is exactly the four cost-accounting patterns of the financial-combinators note.

The dictionary is not a metaphor: the CDT’s *parallel now-frontier* is the combinator **and** (and funds as the cost paper’s atomic join under compound authority); a *holder-controlled branch* is **or** (and funds as the cost paper’s conservative bound with refund); a *nature-resolved branch* (an Obs hole) is the data-dependent demand that the cost paper reserves worst-case and refunds. So the four cost-accounting patterns of the financial note are precisely the four shapes a CDT node can take, and the priced market inherits each pattern’s funding discipline by construction.

Remark 1 (Valuation against a model vs. discovery by market). PJES value a contract by evaluating its denotation against a *model* of the observables. The priced market offers a second route to the same number: it *discovers* the price by crossing bids and asks. $\text{price}(\cdot)$ supplies the buyer’s reservation (a model-based ceiling if it wants one); the clearing engine supplies the realised price. The type guard is admissibility; the quantale-graded **price** is the valuation; the clear is discovery. All three coexist without conflict.

6 Two graded dimensions, kept apart

A bid commits along *two* independent graded axes, and the design’s correctness depends on not fusing them. Price lives in $\mathcal{Q}$ (what the cash flow is worth); phlogiston lives in the resource quantale Φ (how many funded rendezvous it takes to settle). The cost paper is emphatic that these are different: scaling a contract by a large observable multiplies the *value* moved but leaves the *phlogiston* at one — one transfer, one token, whatever its size. Conflating them is a category error.

The generalised acceptance gate. PF grounds when the realised CDT is empty; the cost paper admits a deployment when a linear proof shows $\Sigma_s \geq \Delta_s$ for every signature. The priced

market discharges *both* at once, over the product order:

$$(\Sigma^{\mathcal{Q}}, \Sigma^{\Phi}) \geq_{\mathcal{Q} \times \Phi} (\Delta^{\mathcal{Q}}, \Delta^{\Phi}) \quad \text{component-wise,}$$

where $\Delta^{\mathcal{Q}}$ is the priced demand summed over the grounded tree (by price) and Δ^{Φ} is the phlogiston summed by the located-stack discipline. A grounded priced PF is admitted to run only when it is *both* fully *priced-in* and fully *funded*; the clear handles the first component, the cost gate the second, in a single all-or-nothing reservation.

The seam. The two axes may later be *related* without being merged. The natural connector is a monotone morphism $\kappa : \mathcal{Q} \rightarrow \Phi$ (or a shared numéraire) supplied by an exchange-rate observable — e.g. “the phlogiston budget a deployment may draw is bounded by a function of the value it settles”, or conversely “priority in the clearing rule is purchased in phlogiston”. We name κ as the designed seam and deliberately leave it abstract here: the present design needs only that $\mathcal{Q}$ and Φ are *separate* quantales gated against their product, exactly as the cost paper requires.

7 Bridge: two new folds

The coordination layer carries PFLam terms as payload and already calls four folds ($\models$, grounded, $\triangleleft$, holes). The priced market adds two.

```
// Bridge deltas (rho-level folds delegating to PFLam). pf, ty, q carried as payload.

// val(pf): the PJES demand price of pf over its CDT (sec. 5), in the price
// quantale Q. Pure, total: a conservative reservation the bid can escrow.
Val . pf:Proc |- "val" "(" pf ")" : Proc ![{
    price_proc(crate::pflam::value_over_cdt(&pf))    // -> Price payload
}] fold;

// clear(book, hTy): given the current order book at a hole and the hole type,
// return the matched (ask, clearingPrice) under the venue's clearing RULE,
// among feasible asks {a : a |= hTy and pi_ask <= pi_bid}. The rule is a
// policy parameter (lowest-ask | double-auction | dutch); admissibility is
// always the type guard, never the price.
Clear . book:Proc, hTy:Proc |- "clear" "(" book ", " hTy ")" : Proc ![{
    crate::market::cross(&book, &hTy)                // -> (ask, price) | None
}] fold;
```

`val` is the valuation of §5; `clear` is the only place the clearing policy lives, isolated so the venue can swap auction rules without touching the lifecycle. Note `clear` *cannot* return an ill-typed match: feasibility includes $q \models T$, so the broker’s authority is bounded by the type guard exactly as in PF.

8 Coordination: the matching engine

We extend PF’s protocol. Channels are as in PF, with the demand channel reinterpreted as an order book and three additions:

Channel	Role
@ $[u, \text{“book”}, pfId, h]$	order book at hole h : bids and asks accumulate here
@ $[u, \text{“escrow”}, pfId, h]$	located purse holding the cleared bid’s committed value (& phlo)
@ $[\text{“mkt”}, \text{“prices”}]$	public price registry: cleared prices feed future valuation

8.1 Publish and arm: a market per frontier hole

Publishing walks the CDT as before, but each now-frontier hole is armed as a *market* — seeded with the buyer's bid at the reservation price $\text{price}(h)$, escrowed in a located purse — and each wait-edge is still a subscription that arms the sub-market only when its scrutinee resolves.

```
// armMarket walks one CDT node: open a priced market for each now-hole (in
// PARALLEL), subscribe to each wait-edge (arm sub-market only on the realizing
// outcome). Generalizes PF's armCDT: armDemand becomes armMarket.
contract armMarket(@u, @pfId, @cdt) = {
  match cdt {
    < now: [], wait: [] > => @[u,"ground",pfId]!( *pf )           // realized path empty
    < now: ns, wait: ws > => {
      // (1) frontier: one priced market per now-hole, all concurrent
      for( @[h, ty, pPrice] <- ns ) { openMarket!( u, pfId, h, ty, pPrice ) }
      // (2) edges: subscribe; arm the fiber sub-market on resolve (sec 6.2 of PF)
      | for( @on(s, branches) <- ws ) {
        for( @out <- resolve(*pf, s) ) {
          match out { @[kappa, x] => armMarket!( u, pfId, select(branches, kappa, x) ) }
        }
      }
    }
  }
}

// openMarket: seed the buyer's bid, then run the matching loop for hole h.
contract openMarket(@u, @pfId, @h, @ty, @pBid) = {
  new bidPurse in {
    escrow!( u, pfId, h, pBid, *bidPurse )           // commit value+phlo, all-or-nothing
    | @[u,"book",pfId,h]!!( bid(h, ty, pBid, *bidPurse) ) // buyer's standing demand
    | matchLoop!( u, pfId, h, ty )
  }
}
```

8.2 Offer: a supply PF is one ask plus a bid per residual hole

A seller advertising a holey filler q posts an ask for q and, by the recursion of Def. 2, a bid for each hole in q 's now-frontier. This is the single mechanism that threads price down the tree.

```
// offer(seller, target=(u,pfId,h), q, pAsk): post the ask at the target hole,
// AND open the seller's own buy-side markets for q's residual frontier.
contract offer(@seller, @u, @pfId, @h, @q, @pAsk) = {
  @[u,"book",pfId,h]!!( ask(q, pAsk, addrOf(seller)) ) // sell side at target
  | for( @cdtq <- holes(q) ) {                          // q's own demand tree
    match cdtq { < now: nq, wait: _ > =>
      for( @[hq, tyq, pq] <- nq ) {                      // a BID per residual hole
        openMarket!( seller, q, hq, tyq, pq )           // seller is buyer here
      }
    }
  }
}
```

8.3 Match and clear: atomic, type-guarded, refunding

The matching loop crosses the book under the venue's clearing rule. A clear is one acceptance gate: reserve the clearing price from escrow, pay the seller, plug, record *priced* provenance, refund the surplus, republish.

```
// matchLoop: whenever the book at h admits a feasible crossing, clear it atomically.
contract matchLoop(@u, @pfId, @h, @ty) = {
```



```

for( @book <- @[u,"book",pfId,h] ) {                               // current order book
  match clear(book, ty) {                                         // venue clearing RULE (bridge)
    None => @[u,"book",pfId,h]!!( book )                          // no feasible cross yet; wait
    (ask(q, pAsk, addrA), pStar) => {                             // feasible: pAsk <= pStar <= pBid
      // ---- one acceptance gate: all-or-nothing ----
      for( @bidPurse <- @[u,"escrow",pfId,h] ) {
        reserve!( *bidPurse, pStar, @committed in {             // take clearing price from escrow
          // pay the seller, then settle the structural plug
          settle!( committed, addrA, pStar )                     // value moves seller-ward
        } | for( @pf0 <- @[u,"pf",pfId] ) {
          new pf2 in {
            pf2!( plug(pf0, h, q) ) |                             // structural fill (fires CaseRw if
scrutinee)
          for( @next <- pf2 ) {
            recordProvenance!( u, pfId, h, addrA, pStar ) |      // hole -> (party, price)
            refund!( *bidPurse, sub(pBidOf(book,h), pStar) ) |    // surplus back to buyer
            republish!( u, pfId, next )                           // re-derive CDT: residual frontier +
filler's holes
          }
        }
      }
    }
  }
}
}
}
}
}
}
}
}
}
}

```

The gate is the cost paper's reservation gate verbatim in spirit: **reserve** removes the clearing price from the located escrow purse all-or-nothing; if it fails the purse is untouched and nothing moves; if it succeeds, payment, plug, provenance, and refund all commit together. A broker (matchmaker, reputation service, auctioneer) may pre-rank or pre-filter the book, but cannot cause an ill-typed or over-priced clear, because feasibility is re-checked inside **clear** behind the buyer's escrow.

8.4 Grounding, running, settlement

Grounding is path-realised as in PF: when the realised CDT is empty, the tree is a forest of matched bid/ask pairs each with a committed escrow purse. The generalised gate (§6) then certifies the *whole tree* priced-in and funded before **run** dispatches obligations. Settlement pays each contributing party from the escrow recorded against the hole it filled, on positive attestation; the netting of §2.3 applies wherever a sub-tree is a bilateral swap, collapsing opposed legs into one transfer in the data-dependent direction.

```

// settle: on attestation that party performed step k, release its escrowed payment.
contract settle(@u, @pfId, @k) = {
  for( @[party, price, esc] <- provAt(u, pfId, k) ) {
    for( @ack <- @[u,"step",pfId,k] where ack == "ok" ) {
      for( @purse <- esc ) { transfer!( purse, party, price ) } // pay on positive
attestation
    }
  } | for( @ack <- @[u,"step",pfId,k] where ack != "ok" ) {
    refundAll!( u, pfId ) | fail!( u, pfId, "step-failed" ) // unwind: escrow returns to
buyers
  }
}
}
}
}

```

The crucial liveness improvement over the unpriced design: because each clear escrows value before the plug, a grounded tree cannot strand a contributor who has performed — the funds

for its leg were committed at clear time and are released on attestation, or returned to the buyer on failure. Value is never in flight without a committed purse behind it.

8.5 Templates carry price priors; caveats are price discovery

Success re-abstracts the grounded composite into a template; we additionally record the cleared prices, so a template is a *parametric PF with a price prior* — a structure plus the distribution of clears it has previously fetched, seeding price for the next grounding. Loud failure publishes a caveat; we additionally publish the *prices at which it failed*, so caveats become *price discovery*: the market learns not only “this pattern didn’t work” but “not at this price”. Both feed future valuation through @["mkt", "prices"] monotonically.

9 Lifecycle

The PF lifecycle is preserved; *Matching* is now a clearing state and a new *Escrowed* sub-phase records committed value (Figure 2, Table 2).

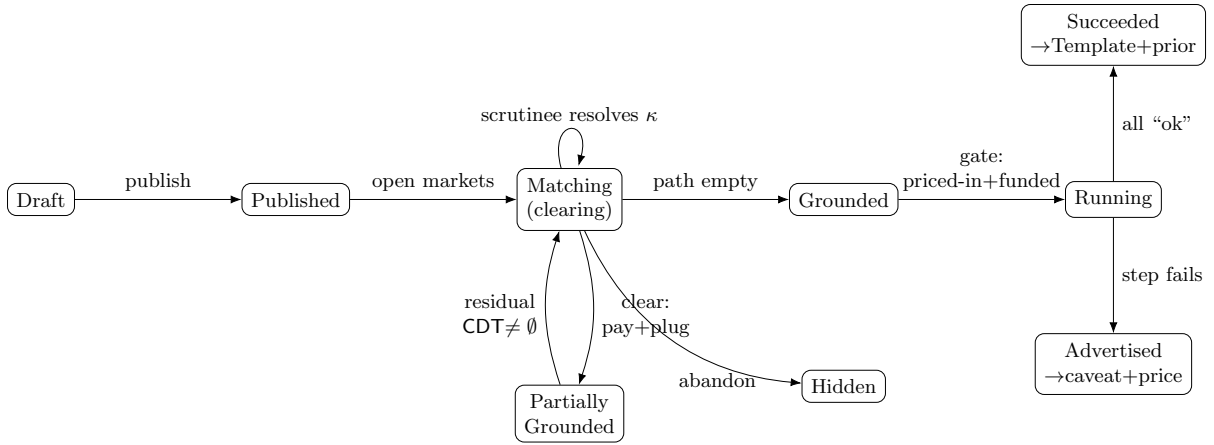


Figure 2: Priced PF lifecycle. Clearing pays and plugs in one gated step; grounding is gated on the whole tree being priced-in *and* funded; success and loud failure now also publish a price (prior / discovery).

From → To	Guard / event	Coordination effect
Draft → Published	publish	store PF; compute CDT; compute price per hole
Published → Matching	now $\neq \emptyset$	open a priced market per frontier hole; escrow bids
Matching → Part. Grounded	clear fires	reserve π_* ; pay seller; plug; record priced prov.; refund $\pi_b - \pi_*$
Matching → Matching	scrutinee resolves κ	arm the κ -fiber’s sub-market
Part. Grounded → Matching	residual CDT $\neq \emptyset$	re-open markets (incl. filler’s holes)
Matching/PG → Grounded	realized path empty	emit @["ground", <i>pfId</i>]
Grounded → Running	gate $\Sigma \geq \Delta$ in $\mathcal{Q} \times \Phi$	dispatch obligations; start monitor
Running → Succeeded	all steps “ok”	settle from escrow; abstract + publish template + price prior
Running → Advertised	step fails (loud)	refund escrow to buyers; publish caveat + failure price
Matching → Hidden	abandon	withdraw bids; release escrow

Table 2: Lifecycle transitions. Differences from PF are the price computation at publish, the pay-and-plug clear, the escrow release on settle/abandon, the product gate at grounding, and the priced template/caveat.

10 Requirements catalogue

We extend PF’s catalogue; only the additions and amendments are listed (numbering continues from PF).

Functional.

FR-12

Each agent hole shall maintain an *order book* of bids and asks at $@[u, \text{“book”}, pfId, h]$, not a single guarded receiver.

FR-13

A *bid* shall carry a type guard T , a reservation price $\pi_b \in \mathcal{Q}$, and a located escrow purse committing value (and phlogiston). Publishing shall seed one bid per now-frontier hole at price $\text{price}(h)$, all concurrently.

FR-14

An *ask* shall carry a filler q with $q \models T$, a reservation price $\pi_a \in \mathcal{Q}$, and a payee address. Posting an ask shall additionally open a bid for each hole in $\text{holes}(q)$ ’s now-frontier (a supply PF is one ask plus a bid per residual hole).

FR-15

$\text{val}(pf)$ shall compute the PJES demand price over the CDT: $\otimes$ over a parallel frontier; conservative bound ($\vee$) over a holder-controlled branch with refund of the unforced side; $o\cdot$ over scale; polarity swap over *give*; option staging over an *Obs* branch (Table 1).

FR-16

A clear shall select, among *feasible* asks ($q \models T$ and $\pi_a \leq_{\mathcal{Q}} \pi_b$), one match at a clearing price $\pi_a \leq \pi_{\star} \leq \pi_b$ under the venue’s clearing rule, and shall perform payment, plug, priced-provenance recording, and surplus refund *atomically* under one acceptance gate.

FR-17

On success the system shall publish a template *with a price prior*; on loud failure a caveat *with the failure price*. Both shall feed val for subsequent groundings.

Non-functional.

NFR-9 *Price $\neq$ phlogiston.* The bid shall commit along two separate graded axes, $\mathcal{Q}$ (price) and Φ (phlogiston); the system shall never identify them and shall gate against their product $\mathcal{Q} \times \Phi$.

NFR-10

Type-bounded brokerage. A broker may rank or pre-filter the book but shall not be able to cause an ill-typed or over-priced clear; feasibility is re-checked behind the buyer’s escrow.

NFR-11

Escrow before plug. No structural plug shall fire before the clearing price is reserved into a committed purse; a performed obligation shall always have committed value behind it, released on attestation or returned to the buyer on failure.

NFR-12

Policy isolation. The clearing rule shall live solely in the **clear** fold, swappable (lowest-ask / double-auction / Dutch) without changing the lifecycle.

NFR-13

Conservative funding. For data-dependent demand the bid shall escrow the worst-case price over branches and the deployment boundary shall refund the unforced remainder (inheriting the cost paper’s over-fund-and-refund discipline).

11 Worked examples: branching, then competition

The priced lifecycle is most easily seen on PF’s branching example. Alice books a venue before she knows indoor vs. outdoor; catering is owed on every branch; a tent is owed iff the venue resolves outdoor, and its type — hence its price — depends on which outdoor venue. Annotating each agent hole with a reservation price, $\text{val}(\text{alicePF})$ is read straight off the CDT: the parallel now-frontier $\{h_{\text{Venue}}, h_{\text{Cater}}\}$ is an atomic join priced $p_V \otimes p_C$, while the wait-edge $\text{on}(h_{\text{Venue}})$ contributes $p_T(v_{\text{out}})$ *only* on the **Outdoor** branch, so the root reservation is $p_V \otimes p_C \otimes (\text{Outdoor}? p_T : 0)$, refunded if Indoor. Publishing arms the venue and catering markets concurrently and escrows their bids; subscribing to $\text{on}(h_{\text{Venue}})$ arms the tent market only once an Outdoor venue is plugged (**CaseRw**), at a price depending on the realised venue; grounding then gates the whole tree priced-in and funded before it runs. This one example exercises *concurrency* (venue and catering clear in parallel), *fibered pricing* (the tent’s price is in the term from the start but escrowed only once Outdoor is realised, so indoor events carry no spurious tent cost), *atomic pay-and-plug*, and *escrow-before-plug*.

What it leaves untested is the *market*. Every hole in it clears against a *single* offered filler, and there is a *single* buyer, so nothing is ever cleared against an alternative and the three gates of §2 — the type guard, the price, and the clearing rule — never pull apart. Appendix A supplies the missing case: the minimal *competitive* configuration, with two holey contexts (two buyers) contending for the same scarce filler and three competing fillers, one of them type-inadmissible, traced through to a clear. It separates the three gates the single-filler case fused — a cheap but ill-typed offer is pruned *before* price is consulted; a richer record is admitted by width subtyping; a single cheap unit is contended by both buyers — and it makes the central honesty of §12 concrete: the clearing *rule* is policy, not a theorem. There the surplus-maximising and coverage-maximising rules *disagree*, and naive greedy concurrency lets the substrate’s associative-commutative nondeterminism choose between them, while located escrow still keeps a stranded buyer *safe* (fully refunded) — separating safety from liveness. The full setup, feasibility graph, side-by-side clearing-policy comparison, and concurrent trace are worked in Appendix A.

12 Established vs. conjectural; open problems

In the spirit of both source documents we separate what the fusion gets from what it hopes for.

Established (given a normalising PFLam fragment and the cost gate).

- *Admissibility is unchanged and decidable.* Priced annotates a hole without touching its type, so $q \models T$ and all of PF’s decidability/termination results carry over verbatim; pricing never enters the acceptance test.
- *val is total and structural.* On the normalising, recursion-free fragment the CDT is finite, so price is a terminating fold; the four clauses are exactly the four cost-accounting patterns and inherit their funding discipline.
- *Each clear is atomic.* Payment, plug, provenance, and refund commit under one acceptance gate; this is the cost paper’s reservation gate, and ill-typed/over-priced clears are excluded by re-checking feasibility behind escrow.

- *Grounded trees are funded before they run.* The product gate $\Sigma \geq \Delta$ in $\mathcal{Q} \times \Phi$ lifts the cost paper’s linear resource proof from one step to the whole tree; escrow-before-plug means no value is ever in flight without a committed purse.
- *Netting is clearing.* A bilateral swap (give-paired legs) clears as a single net transfer, recovering the cost paper’s netting as the market’s ordinary operation (§2.3).

Conjectural / delegated / next.

- *The clearing rule is policy, not theorem.* Lowest-ask, uniform-price double auction, and Dutch descent are all admissible; which one the venue runs — and whether it is incentive-compatible, budget-balanced, or strategy-proof — is mechanism design, out of scope for the calculus. The calculus guarantees only that whatever rule `clear` encodes settles atomically and type-safely.
- *Pricing a nature-resolved branch wants a probability layer.* The conservative bound ($\bigvee$ over branches) is the distribution-free ceiling; the sharp price of an `Obs` branch is an expectation $\mathbb{E}_\mu[\cdot]$ under a model μ of the outcome. Supplying μ , and letting the market *discover* it, is precisely a prediction-market layer over the same CDT — the natural next venue, and the point at which `val` stops being a fold and becomes a forecast.
- *The price/phlogiston seam κ is named, not built.* We require only that $\mathcal{Q}$ and Φ are separate and gated against their product; a monotone $\kappa : \mathcal{Q} \rightarrow \Phi$ (or shared numéraire) connecting them — value-bounded gas, or priority bought in phlogiston — is a deliberate later decision.
- *Priced templating is anti-unification over prices too.* Re-abstracting a success must now generalise the cleared *prices* as well as the structure; the principled construction is least general generalisation over a typed price-annotated term, whose exact order (and how a price prior is maintained as a distribution rather than a point) is open — the same lgg subtlety PF already flags, now graded.
- *Liveness is still a market property.* “Every priced PF eventually clears and grounds” depends on participation, liquidity, and incentives; it is not a theorem. Escrow improves *safety* (no stranded contributor) but not *liveness* (a market with no feasible ask never clears).
- *Execution remains off-chain and attested.* As in both papers, world-effecting steps are confirmed by signed oracle attestations; the escrow-release-on-attestation design inherits the cost paper’s oracle trust model and its open dispute-resolution question, now with funds at stake.

Suggested next decisions. (1) Fix the clearing rule and its mechanism-design properties. (2) Choose the price quantale $\mathcal{Q}$ concretely and the play-payoff $\text{val} : \text{Runs}(\text{CDT}) \rightarrow \mathcal{Q}$, including the preference (“earlier is better”) instances. (3) Add the probability layer for `Obs` branches, turning conservative bounds into expectations and opening the prediction market. (4) Decide the seam κ relating price to phlogiston. (5) Specify priced templating (price-prior maintenance; lgg over price-annotated terms). (6) Design the escrow dispute-resolution and where economic penalties attach on failed attestation.

A A Worked Example with Competition

The worked example in the main text (Alice’s venue) exhibits *branching* and *fibred demand*, but it never exercises a *market*: each hole sees at most one ask, and there is only one buyer, so nothing is ever *cleared* against an alternative. This appendix builds the minimal configuration

that does: **two competing holey contexts** (two buyers, Alice and Dave, each with one venue hole) and **three competing fillers** (Carol, Bob, and Eve), one of which is type-inadmissible. The example is small enough to trace by hand and rich enough to separate three gates that the single-filler example fused: *type pruning* (Eve, the cheapest offer, never competes because she fails the type guard), *subtyping admission* (Bob’s richer record satisfies the leaner demand), and *price clearing of scarce supply* (one cheap venue, Carol, contended by both buyers). The payoff is a concrete demonstration of the main text’s central honest claim (§12): the clearing *rule* is policy, not a theorem. We show that the surplus-maximising rule and the coverage-maximising rule *disagree* on this five-order example, and that naive greedy concurrency lets the substrate’s associative–commutative nondeterminism silently choose between them — whichever book grabs Carol first decides which buyer is served. Escrow keeps the stranded buyer *safe* (full refund) even though the market leaves him *unserved*: a clean separation of safety from liveness.

A.1 What the Alice example did not show

The main text’s worked example (Alice’s venue, indoor/outdoor tent contingency) is the right vehicle for the CDT: it shows a now-frontier armed in parallel, a wait-edge fibering the tent demand under the **Outdoor** outcome, and a price that depends on the realised branch. But every hole in it clears against a *single* offered filler, and there is a *single* buyer. The two-sided machinery — an order book with competing asks, two books contending for one scarce supply, a clearing rule that must *choose* — is never put under load. A “truly worked” example must construct competition on both sides and run it to a clear. That is what this appendix supplies.

The three gates we want to separate. The main text insists (§2, §5, Table 1) that matching is *admissibility*, valuation is the *quantitative companion*, and clearing is *discovery*. With one ask per hole these collapse into a single “plug it”. With several asks they pull apart into a pipeline that the trace below makes visible:

$$\underbrace{q \models T}_{\text{type gate: admit}} \longrightarrow \underbrace{\pi_a \leq_Q \pi_b}_{\text{price gate: feasible}} \longrightarrow \underbrace{\text{clearing rule selects, atomically.}}_{\text{rule gate: allocate scarce supply}}$$

A.2 Setup: one type, two buyers, three fillers

A.2.1 The demanded type

A venue is a dependent record (records-as- Σ , main text §4): two fields, capacity and an outdoor flag.

$$\text{Venue} \equiv \text{Sig}(cap : \text{Nat}). \text{Sig}(outdoor : \text{Bool}). \text{Unit}, \quad \{cap=n, outdoor=b\} : \text{Venue}.$$

We use Σ -record *width subtyping*: a record with *more* fields is a subtype of one with fewer (a richer offer satisfies a leaner demand — the main text’s phrase, §4). This is the only subtyping the example needs, and it makes admissibility a decidable structural check, never a search.

A.2.2 Two competing holey contexts (the buyers)

Each buyer publishes a PFLam term with exactly *one* agent hole of type **Venue** — minimal, so the book at that one hole is the whole story. They differ only in reservation price.

```
// Buyer A = Alice. One hole hV : Venue. Reservation price 100.
alicePF = reqs . book (? hV : Venue @ 100) caterA

// Buyer B = Dave. One hole hV' : Venue. Reservation price 70.
davePF  = reqs . book (? hV' : Venue @ 70) caterD
```

Both holes demand the *same* type, `Venue`, so they draw on the *same* pool of venue fillers. That shared dependence is what makes the two contexts *compete*: a venue plugged into one is a venue unavailable to the other.

A.2.3 Three competing fillers (the asks)

Three sellers advertise venues. Each holds *one* unit (scarce supply), offered on its own located supply channel.

Seller	Offered filler q	Ask price π_a	Shape vs. Venue
Carol	$\{cap=90, outdoor=true\}$	60	exact match
Bob	$\{cap=120, outdoor=true, parking=true\}$	80	<i>wider</i> record ($<$: Venue by width)
Eve	$\{seats=200\}$	10	<i>wrong</i> shape ($\not<$: Venue: no <i>cap</i> , no <i>outdoor</i>)

Eve is deliberately the *cheapest* offer in the market. If price ranked before type, Eve would win every hole. She does not, because she never enters a feasible set: the type guard prunes her first. This is the cleanest possible statement of “the type system is the acceptance test, not the search” (main text §2).

A.3 The feasible set: admit, then price

Type gate (admit). Carol $\models$ Venue (exact) and Bob $\models$ Venue (Bob’s $\{cap, outdoor, parking\} <: \{cap, outdoor\}$ by width subtyping). Eve $\not\models$ Venue: $\{seats\}$ has neither field, so no synthesised type subtypes Venue. **Eve is pruned**, at any price.

Price gate (feasible). Among the admissible asks $\{\text{Carol } 60, \text{Bob } 80\}$, an ask is *feasible* for a bid iff $\pi_a \leq \pi_b$:

Alice ($\pi_b=100$) : $\{\text{Carol}, \text{Bob}\}$ ($60 \leq 100, 80 \leq 100$); Dave ($\pi_b=70$) : $\{\text{Carol}\}$ ($60 \leq 70, 80 \not\leq 70$).

Bob is admissible but *infeasible* for Dave (Bob asks more than Dave will pay). The feasibility graph is bipartite, with each edge weighted by its surplus $\pi_b - \pi_a$ (Figure 3).

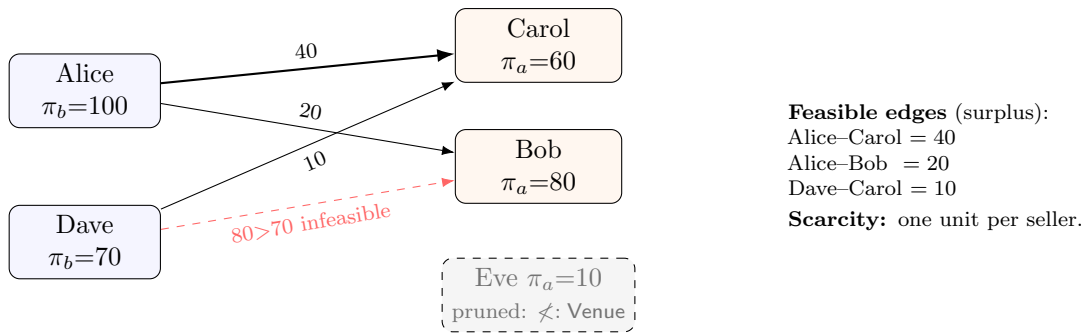


Figure 3: The feasibility graph after the type gate (Eve removed) and the price gate (Dave–Bob removed). Edge weights are surpluses $\pi_b - \pi_a$. The whole market turns on a single contended unit: Carol, the cheapest admissible venue, sits on *both* buyers’ only-or-best edge.

A.4 The books as data

Each buyer’s hole channel holds an order book: its own standing bid plus every forwarded ask. (Brokers forward all three asks to both books; the type guard inside `clear` re-prunes Eve, so a broker cannot smuggle her in — NFR-10.)

```

// Alice's book at hV (escrow purse L_A holds 100 committed):
@[alice,"book","ev",hV] = [ bid(hV, Venue, 100, L_A),
                             ask(carolVenue, 60, addrCarol),
                             ask(bobVenue, 80, addrBob),
                             ask(eveHall, 10, addrEve) ] // eve present but NOT a Venue

// Dave's book at hV' (escrow purse L_D holds 70 committed):
@[dave,"book","ev",hV'] = [ bid(hV', Venue, 70, L_D),
                             ask(carolVenue, 60, addrCarol),
                             ask(bobVenue, 80, addrBob),
                             ask(eveHall, 10, addrEve) ]

// Scarce supply: ONE located unit per seller. clear() consumes it once.
@L_carol = [ unit(carolVenue) ] // a single located stack (cost paper sec 3.2)
@L_bob   = [ unit(bobVenue) ]
@L_eve   = [ unit(eveHall) ]

```

Carol's single located unit $@L_{carol}$ is the disjoint token pool of the cost paper's §8.8: two books contending for it are two competing linear-proof obligations, of which *at most one* succeeds. The next two sections show that the answer to “which one” is not decided by the calculus.

A.5 Clearing: the rule decides who is served

$\text{clear}(\text{book}, \text{Venue})$ returns a match among *feasible* asks under the venue's policy. Five reasonable policies give three distinct outcomes; Table 3 lays them side by side.

Clearing rule	Allocation (price paid)	Served	Surplus
Greedy lowest-ask, <i>Alice's book fires first</i>	Alice→Carol (60); stranded; Bob unsold	1/2	40
Greedy lowest-ask, <i>Dave's book fires first</i>	Dave→Carol (60); Alice→Bob (80)	2/2	30
Uniform-price double auction	Alice→Carol (≈ 75); Dave excluded; Bob unsold	1/2	40
Global <i>surplus-maximising</i> matching	Alice→Carol (60); stranded	1/2	40
Global <i>coverage-maximising</i> matching	Alice→Bob (80); Dave→Carol (60)	2/2	30

Table 3: Five clearing policies, three outcomes. Surplus = $\sum(\pi_b - \pi_a)$ over cleared trades; “served” counts buyers grounded. The two *global, principled* rules disagree: maximising surplus strands Dave (one rich trade, 40); maximising coverage serves both at lower total surplus (30). No rule is forced by the calculus.

The surplus/coverage fork. This is the example's point. The two “correct” global objectives select *different* allocations:

- *Maximise surplus.* The single edge Alice–Carol (weight 40) beats any two-edge matching ($20+10 = 30$). So surplus-max plugs Carol into Alice and **strands Dave** — the most valuable single trade leaves a willing buyer unserved.
- *Maximise coverage.* Serving both buyers forces Alice onto Bob and Dave onto Carol (**the only** way to clear two units), total surplus 30. Alice pays 80 instead of 60 — coverage costs her 20.

There is no calculus-level reason to prefer 40-with-one-served over 30-with-both-served; it is a stated preference of the venue (efficiency vs. inclusion), exactly the “valuation is the next layer... not a type” point of the main text’s §12. The price quantale $\mathcal{Q}$ ranks individual asks; it does *not* by itself pick the social objective over the whole book.

Concurrency silently chooses the objective. Worse — and this is why NFR-12 (policy isolation) is load-bearing rather than tidy — if the venue runs *no* global rule but just lets each book greedily take its cheapest feasible ask (the literal reading of the main text’s per-hole `matchLoop`), then both books race for Carol, and the associative-commutative soup picks a winner nondeterministically:

Alice grabs Carol first $\Rightarrow$ surplus-max outcome (40, 1/2),
 Dave grabs Carol first $\Rightarrow$ coverage-max outcome (30, 2/2).

The scheduler, not the designer, ends up choosing the welfare criterion. The lesson the example teaches is therefore sharper than “pick a rule”: *not picking one does not avoid the choice, it delegates it to the runtime.*

A.6 The concurrent trace (greedy lowest-ask)

We trace the greedy reading to show the atomic single-consume and the safety of the stranded buyer. Both books reach the same cheapest feasible ask, Carol, and both attempt to consume her one located unit.

```
// Each matchLoop, having computed cheapest-feasible = Carol, attempts the consume.
// The located unit @L_carol can be received ONCE; the AC soup picks the winner.
contract tryClear(@u, @pfId, @h, @ask(q, pAsk, addrA), @Lsupply, @bidPurse, @pBid) = {
  for( @unit <- @Lsupply ) { // <-- single-consume; only ONE book wins
    // winner path: one acceptance gate (reserve, pay, plug, refund), all-or-nothing
    reserve!( *bidPurse, pAsk, @committed in {
      settle!( committed, addrA, pAsk ) // pay seller pAsk
    } | for( @pf0 <- @[u,"pf",pfId] ) { new pf2 in {
      pf2!( plug(pf0, h, q) ) |
      for( @next <- pf2 ) {
        recordProvenance!( u, pfId, h, addrA, pAsk ) |
        refund!( *bidPurse, sub(pBid, pAsk) ) | // surplus back to buyer
        republish!( u, pfId, next ) // residual CDT empty -> GROUND
      } } }
  })
}
// loser path: @L_carol is now empty, so the for(...) above never fires for the loser;
// the loser's matchLoop re-reads its book WITHOUT Carol and recomputes feasibility.
}
```

Say Alice wins Carol. Alice’s gate reserves 60 from L_A , pays Carol, plugs $hV \leftarrow \text{carolVenue}$, refunds 40, republishes; her residual CDT is $\langle \text{now} : [], \text{wait} : [] \rangle$, so **Alice grounds**. Dave’s consume of $@L_carol$ never fires (the unit is gone); his `matchLoop` re-reads its book, now $\{\text{Bob}\}$ among the admissible, and finds Bob *infeasible* ($80 > 70$). Dave’s feasible set is *empty*: **Dave does not ground**. He stays Matching and, on the abandon timeout, retires to Hidden; his escrow L_D was *never touched*, so his 70 is returned in full.

The alternate order (Dave wins Carol). Dave grounds at 60 (surplus 10). Alice’s `matchLoop` re-reads its book without Carol, finds Bob feasible ($80 \leq 100$), and clears Bob at 80 (surplus 20); both buyers ground. Same inputs, different interleaving, different served set and different price for Alice — the nondeterminism of §A.5 made operational.

Remark 2 (Safety $\neq$ liveness, made concrete). In the Alice-wins order Dave is left unserved. This is a *liveness* shortfall (no feasible ask remained for him), *not* a *safety* violation: because escrow precedes every plug (NFR-11), Dave’s committed value sat untouched on L_D and is refunded on abandon. No counterparty is ever half-paid, no unit double-allocated (Carol’s located stack is consumed once), and no value strands. The market can fail to *match* a buyer; it cannot *lose* his funds. This is precisely the separation the main text claims (§12, “escrow improves safety... not liveness”).

A.7 Settlement

In any grounding order, settlement dispatches the grounded buyers’ obligations from priced provenance and releases escrow on attestation. For the coverage outcome $\{\text{Alice} \rightarrow \text{Bob}@80, \text{Dave} \rightarrow \text{Carol}@60\}$: on Bob’s and Carol’s signed attestations, 80 releases from L_A to Bob and 60 from L_D to Carol; the unspent $\pi_b - \pi_\star$ (20 for Alice, 10 for Dave) was already refunded at clear time. Bob’s and Carol’s deployments are signed by *different* signatures over *disjoint* escrow pools, so they settle under concurrent acceptance with no merge analysis (cost paper §8.8): the two trades neither block nor interfere.

A.8 What this example establishes

Mapping the trace back onto the main text’s claims and requirements:

- **Type prunes before price** (§2, FR-16). Eve, the cheapest offer at 10, never enters a feasible set; admissibility is logically and operationally prior to the price ranking. A market that ranked price first would mis-clear here.
- **Subtyping admits the richer offer** (§4, FR-14). Bob’s three-field record satisfies the two-field demand by width subtyping — “a richer offer satisfies a leaner demand” is exercised, not just asserted.
- **Competing fillers make the clearing rule bite** (§2, FR-16, NFR-12). With two admissible feasible asks at Alice’s hole, `clear` must *select*; lowest-ask vs. uniform-price already diverge on what Alice pays.
- **Competing contexts make allocation a policy** (§12). Two buyers and one cheap unit force a scarce-supply allocation; surplus-max and coverage-max give *different* answers, so no rule is calculus-forced. This is the example’s headline.
- **Disjoint escrow = safe concurrency** (§8, NFR-11, cost paper §8.8). Carol’s single located unit is consumed once; competing books are competing linear-proof obligations, at most one succeeds; the loser’s escrow is intact.
- **Safety without liveness** (§12). The stranded buyer is fully refunded; failure to match is not failure to protect funds.

Which open problems it makes concrete. The example is the smallest witness to two of the main text’s deferred items. (1) *The clearing rule is mechanism design* (§12, “policy, not theorem”): here even the choice between two principled global rules has no calculus-level answer, and greedy concurrency delegates it to the scheduler — so the venue *must* fix a rule deliberately (and, if it wants the coverage outcome, run a *global* two-sided clear over both books, i.e. a max-weight or max-cardinality matching on Figure 3, not the per-hole `matchLoop`). (2) *Valuation is a layer above the type* (§12): the social objective (efficiency vs. coverage) is a preference over allocations, not a predicate and not a price on any single ask — exactly the `val : Runs  $\rightarrow$  Q` object the main text names and leaves open.

A note on the right venue design. To realise the coverage outcome deterministically, the `matchLoop` of the main text (greedy, per hole) is insufficient: it must be replaced by a *batched two-sided clear* that reads both books, builds the feasibility graph (Figure 3), and runs the chosen matching objective before committing any plug. The located-escrow discipline is unchanged — each matched edge still clears under one acceptance gate over its own purse — but the *selection* is global. This is the concrete shape of the “double-auction” clearing rule named in NFR-12, and the point at which the prediction-market / mechanism-design layer (§12) attaches.

Glossary

priced PF	A PFLam term whose agent holes carry reservation prices in $\mathcal{Q}$; the buyer’s object is the term plus a bid per frontier hole.
bid	$(\Gamma_h \vdash T, \pi_b, L_b)$: a buyer’s demand at a hole — type guard, reservation price, located escrow purse. “PF + a cost”.
ask	$(q, \pi_a, addr)$ with $q \models T$: a seller’s offer at a hole; posting it also opens a bid per residual hole of q . A “PF whose holes are priced”.
order book	The per-hole accumulation of bids and asks at $@[u, \text{“book”}, pfId, h]$; replaces PF’s single guarded receiver.
feasible	An ask feasible for a bid: $q \models T$ (type-admissible) and $\pi_a \leq_{\mathcal{Q}} \pi_b$ (price-compatible).
clear	The atomic selection of a feasible ask at $\pi_\star$ under the venue’s clearing rule, with payment, plug, priced provenance, and surplus refund under one gate.
clearing rule	The policy (lowest-ask / double-auction / Dutch) isolated in the <code>clear</code> fold; not forced by the calculus.
val / price	The PJES demand price over the CDT, valued in $\mathcal{Q}$; admissibility’s quantitative companion.
$\mathcal{Q}$	The preference/price quantale; $\otimes$ combines concurrent demand, $\leq$ ranks.
Φ	The phlogiston (resource) quantale; kept strictly separate from $\mathcal{Q}$.
κ	The named (unbuilt) seam $\mathcal{Q} \rightarrow \Phi$ relating price to phlogiston.
escrow	The located purse committing a cleared bid’s value (and phlo) before any plug fires; released on attestation, returned to the buyer on failure.
price prior	The clearing-price distribution a successful template carries, seeding <code>val</code> for re-grounding.
netting-as-clearing	A bilateral swap clears as one net transfer; the cost paper’s netting is the market’s ordinary clear (§2.3).